LAPORAN PRAKTIKUM BASIS DATA
MODUL 10: DBMS DAN APLIKASI BASIS DATA

Nama	:	KHANSA QAIS FERNANDA
NIM	:	264311019
Kelas	:	TRPL 1A
Kelompok	:	KELOMPOK MAC OS
Anggota & Skor	:	1. Khansa Qais Fernanda (264311019) - Skor: 4 (Sangat Baik) 2. Jelita Palupi (264311017) - Skor: 4 (Sangat Baik) 3. Zahro Choirun Latifah (264311030) - Skor: 4 (Sangat Baik)
Studi Kasus	:	Implementasi Basis Data Penjualan Sederhana (praktikum_modul10)
DBMS	:	MySQL Community Server 8.4.3 (InnoDB) via Laragon / phpMyAdmin / MySQL CLI

10.2 SOAL PRE-TEST

1. Jelaskan apa yang dimaksud dengan DBMS dan apa perbedaannya dengan aplikasi pengelola basis data seperti phpMyAdmin atau MySQL Workbench! [Bobot: 30]

JAWAB:

- Pengertian DBMS: Database Management System (DBMS) adalah perangkat lunak inti (database server) yang bertugas menyimpan data secara fisik, memproses perintah SQL, mengelola transaksi (ACID), mengatur hak akses, serta menjaga integritas dan pemulihan data. Contohnya adalah MySQL Server (mysqld.exe), MariaDB, dan PostgreSQL.

- Perbedaan dengan Aplikasi Pengelola: DBMS adalah mesin penyimpan dan pemroses data yang berjalan di belakang layar (misalnya di port 3306), sedangkan aplikasi pengelola basis data (Database Management Client) seperti phpMyAdmin, MySQL Workbench, HeidiSQL, dan DBeaver hanyalah antarmuka visual (GUI/Web) untuk memudahkan pengguna mengirim perintah SQL ke DBMS.

2. Apa fungsi Primary Key dan Foreign Key dalam menjaga integritas data pada basis data relasional? [Bobot: 30]

JAWAB:

- Fungsi Primary Key (PK): Menjaga integritas entitas (Entity Integrity) dengan memastikan setiap baris data di dalam tabel memiliki identitas yang unik dan tidak boleh kosong (NOT NULL), sehingga tidak ada data induk yang ganda.

- Fungsi Foreign Key (FK): Menjaga integritas referensial (Referential Integrity) antar tabel yang berelasi dengan memastikan nilai kunci tamu pada tabel anak selalu merujuk pada nilai Primary Key yang sah di tabel induk. FK mencegah transaksi merujuk ke data induk yang tidak ada (orphan record) dan mencegah data induk dihapus sembarangan saat masih memiliki riwayat transaksi.

3. Mengapa penggunaan indeks dapat membantu meningkatkan performa query, dan apakah semakin banyak indeks selalu membuat basis data semakin cepat? Jelaskan! [Bobot: 40]

JAWAB:

- Alasan Indeks Meningkatkan Performa Query: Indeks membuat struktur pencarian terurut (B-Tree) pada kolom tertentu sehingga DBMS bisa langsung melompat menemukan lokasi baris yang dicari tanpa harus memeriksa seluruh isi tabel satu per satu dari awal sampai akhir (Full Table Scan).

- Apakah Semakin Banyak Indeks Selalu Lebih Cepat? Tidak. Terlalu banyak indeks justru memperlambat operasi tambah, ubah, dan hapus data (INSERT, UPDATE, DELETE) karena setiap kali ada perubahan

data di tabel, DBMS harus ikut memperbarui seluruh pohon indeks yang ada serta memakan ruang penyimpanan tambahan.

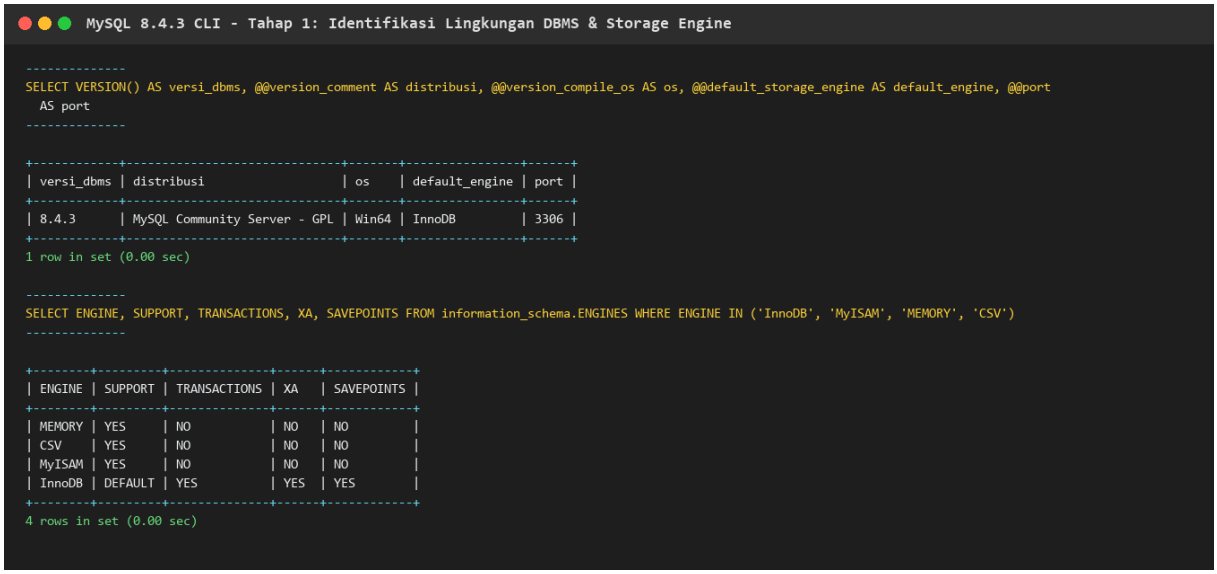
10.3 TUGAS / UNJUK KERJA (PRAKTIKUM & PRESENTASI)

1. Eksplorasi DBMS dan Arsitektur Sistem (Tahap 1) [Skor: 25]

JAWAB:

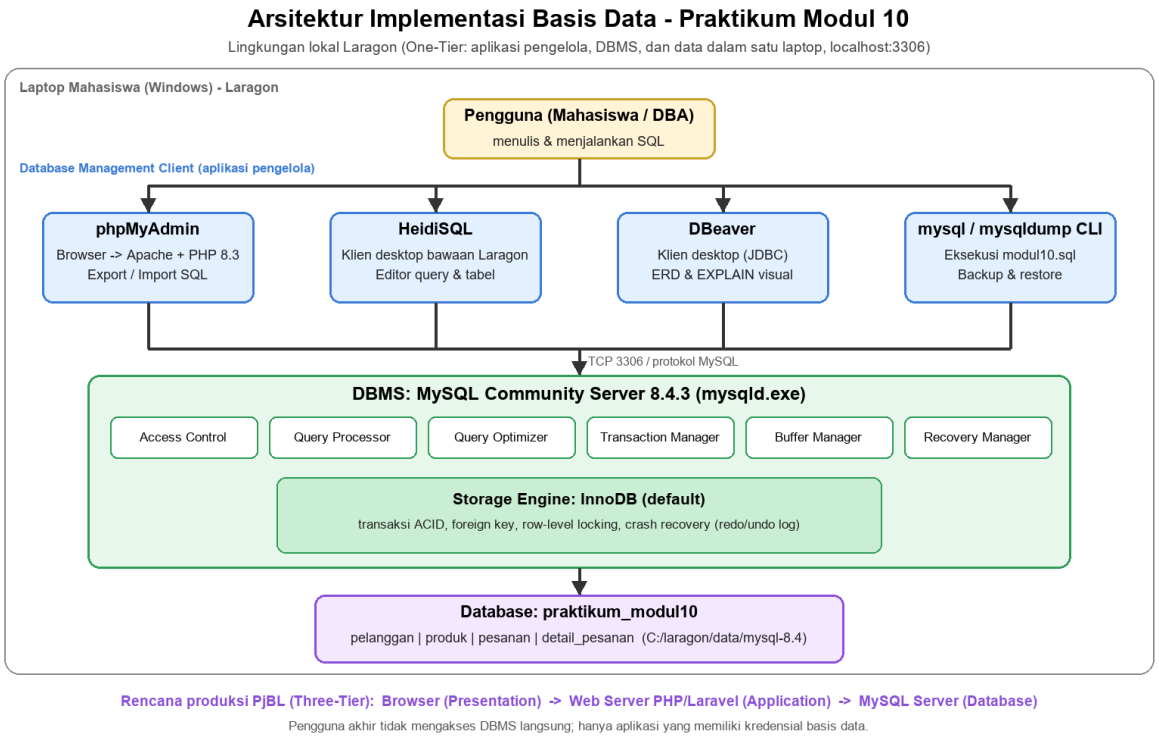
Kami menjalankan perintah SQL untuk mengidentifikasi versi DBMS, vendor distribusi, dan storage engine yang aktif pada mesin praktikum:

```
SELECT VERSION() AS versi_dbms, @@version_comment AS distribusi, @@version_compile_os AS os, @@default_storage_engine AS default_engine, @@port AS port;
SELECT ENGINE, SUPPORT, TRANSACTIONS, XA, SAVEPOINTS FROM information_schema.ENGINES WHERE ENGINE IN ('InnoDB', 'MyISAM', 'MEMORY', 'CSV');
```



Gambar 1. Hasil Identifikasi Versi DBMS (MySQL 8.4.3) dan Storage Engine InnoDB

- DBMS yang Digunakan: MySQL Community Server 8.4.3 (Win64) berjalan pada port TCP 3306.
- Storage Engine: InnoDB (Default), karena mendukung penuh transaksi ACID, Foreign Key, dan row-level locking.
- Aplikasi Pengelola: Kombinasi phpMyAdmin / HeidiSQL / DBeaver (antarmuka visual) serta MySQL CLI (mysql.exe & mysqldump.exe untuk eksekusi batch jutaan baris dan backup/restore cepat).
- Arsitektur Sistem: Lingkungan praktikum menggunakan One-Tier Architecture (aplikasi pengelola, server MySQL 8.4.3, dan penyimpanan data berjalan lokal di satu laptop mahasiswa melalui Laragon localhost:3306). Untuk proyek PjBL berbasis web digunakan Three-Tier Architecture (Browser -> Web Server Aplikasi -> MySQL Server).



Gambar 2. Diagram Arsitektur Implementasi Sistem Basis Data (arsitektur_db.png)

2. Implementasi Skema Basis Data (Tahap 2) [Skor: 30 - Bagian A]

JAWAB:

Kami membangun database praktikum_modul10 dengan 4 tabel yang saling berelasi menggunakan storage engine InnoDB (pelanggan, produk, pesanan, dan detail_pesanan):

```
CREATE DATABASE praktikum_modul10 CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE praktikum_modul10;

CREATE TABLE pelanggan (
    id_pelanggan INT AUTO_INCREMENT PRIMARY KEY,
    nama_pelanggan VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    no_hp VARCHAR(20),
    kota VARCHAR(50) NOT NULL,
    tanggal_daftar DATE NOT NULL DEFAULT (CURRENT_DATE)
) ENGINE=InnoDB;

CREATE TABLE produk (
    id_produk INT AUTO_INCREMENT PRIMARY KEY,
    nama_produk VARCHAR(100) NOT NULL,
    kategori VARCHAR(50) NOT NULL,
    harga DECIMAL(12,2) NOT NULL,
    stok INT NOT NULL DEFAULT 0,
    CONSTRAINT chk_produk_harga CHECK (harga >= 0),
    CONSTRAINT chk_produk_stok CHECK (stok >= 0)
) ENGINE=InnoDB;

CREATE TABLE pesanan (
    id_pesanan INT AUTO_INCREMENT PRIMARY KEY,
    id_pelanggan INT NOT NULL,
    tanggal_pesanan DATETIME NOT NULL,
    status ENUM('baru', 'dibayar', 'dikirim', 'selesai', 'batal') NOT NULL DEFAULT 'baru',
```

```
    total_harga DECIMAL(14,2) NOT NULL DEFAULT 0,
    CONSTRAINT fk_pesanan_pelanggan FOREIGN KEY (id_pelanggan)
        REFERENCES pelanggan (id_pelanggan)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB AUTO_INCREMENT=101;

CREATE TABLE detail_pesanan (
    id_detail INT AUTO_INCREMENT PRIMARY KEY,
    id_pesanan INT NOT NULL,
    id_produk INT NOT NULL,
    jumlah INT NOT NULL,
    harga_saat_transaksi DECIMAL(12,2) NOT NULL,
    CONSTRAINT uq_detail_pesanan_produk UNIQUE (id_pesanan, id_produk),
    CONSTRAINT chk_detail_jumlah CHECK (jumlah > 0),
    CONSTRAINT chk_detail_harga CHECK (harga_saat_transaksi >= 0),
    CONSTRAINT fk_detail_pesanan FOREIGN KEY (id_pesanan)
        REFERENCES pesanan (id_pesanan)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_detail_produk FOREIGN KEY (id_produk)
        REFERENCES produk (id_produk)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;
```



Gambar 3. Pemeriksaan Tabel, Storage Engine InnoDB, dan Relasi Foreign Key

3. Praktik Operasi CRUD dan Pengujian Integritas Referensial (Tahap 3) [Skor: 30 - Bagian B]

JAWAB:

```
INSERT INTO pelanggan (nama_pelanggan, email, no_hp, kota) VALUES
('Andi Pratama', 'andi@mail.test', '081200000001', 'Surabaya'),
```

```
('Budi Santoso', 'budi@mail.test', '081200000002', 'Malang'),
('Citra Lestari', 'citra@mail.test', '081200000003', 'Sidoarjo'),
('Dewi Anggraini', 'dewi@mail.test', '081200000004', 'Surabaya'),
('Eka Saputra', 'eka@mail.test', '081200000005', 'Gresik');

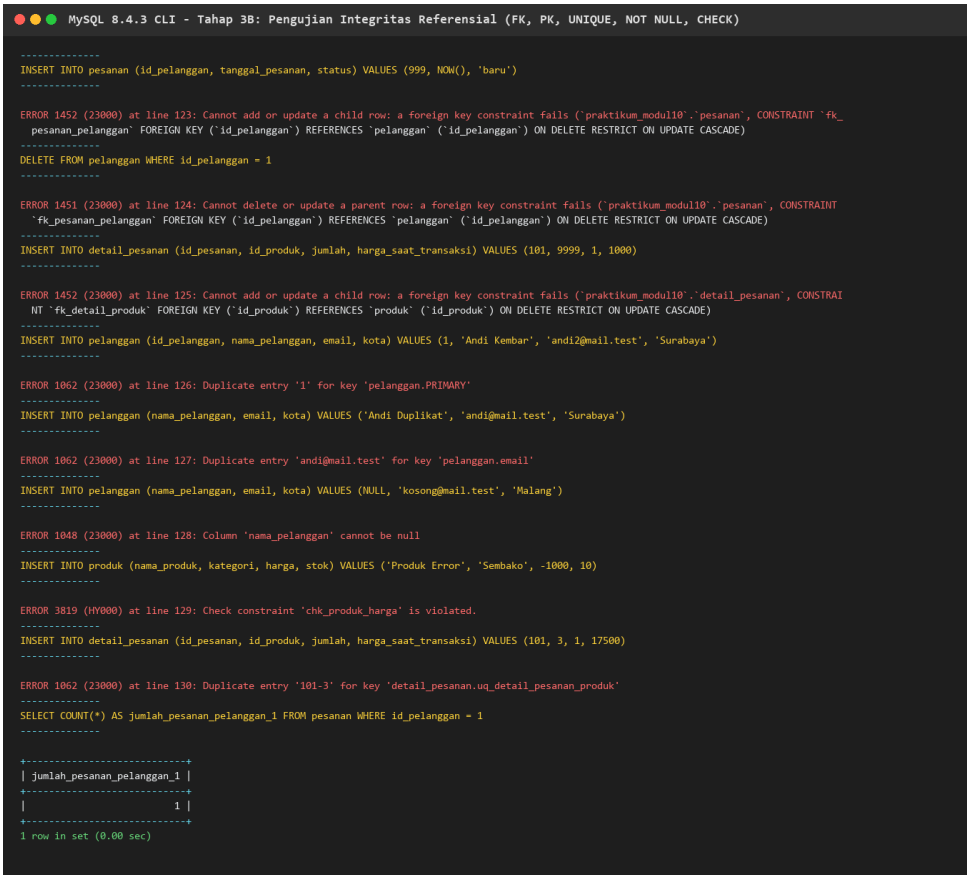
SELECT ps.id_pesanan, pl.nama_pelanggan, pr.nama_produk, dp.jumlah, dp.harga_saat_transaksi,
       dp.jumlah * dp.harga_saat_transaksi AS subtotal
FROM pesanan ps
JOIN pelanggan pl ON pl.id_pelanggan = ps.id_pelanggan
JOIN detail_pesanan dp ON dp.id_pesanan = ps.id_pesanan
JOIN produk pr ON pr.id_produk = dp.id_produk
ORDER BY ps.id_pesanan, pr.nama_produk;

UPDATE produk SET harga = 80000 WHERE id_produk = 1;
DELETE FROM pelanggan WHERE id_pelanggan = 6;

INSERT INTO pesanan (id_pelanggan, tanggal_pesanan, status) VALUES (999, NOW(), 'baru');
DELETE FROM pelanggan WHERE id_pelanggan = 1;
```



Gambar 4. Hasil Eksekusi Operasi CRUD (CREATE, READ JOIN, UPDATE, dan DELETE)



Gambar 5. Bukti Penolakan Data Tidak Valid oleh Constraint FK, PK, UNIQUE, NOT NULL, dan CHECK

4. Pengujian Transaksi Database COMMIT & ROLLBACK (Tahap 4) [Skor: 25 - Bagian A]

JAWAB:

```
START TRANSACTION;
INSERT INTO pesanan (id_pesanan, id_pelanggan, tanggal_pesanan, status) VALUES (103, 3, '2026-10-03 09:30:00', 'dibayar');
INSERT INTO detail_pesanan (id_pesanan, id_produk, jumlah, harga_saat_transaksi) VALUES (103, 4, 2, 25000), (103, 6, 3, 4500);
UPDATE produk SET stok = stok - 2 WHERE id_produk = 4;
UPDATE produk SET stok = stok - 3 WHERE id_produk = 6;
COMMIT;

START TRANSACTION;
INSERT INTO pesanan (id_pesanan, id_pelanggan, tanggal_pesanan, status, total_harga) VALUES (104, 4, '2026-10-04 11:00:00', 'baru', 80000);
INSERT INTO detail_pesanan (id_pesanan, id_produk, jumlah, harga_saat_transaksi) VALUES (104, 1, 1, 80000);
ROLLBACK;
```



Gambar 6. Bukti Pengujian Transaksi COMMIT (Pesanan 103 Tersimpan) dan ROLLBACK (Pesanan 104 Dibatalkan)

- Perbedaan COMMIT & ROLLBACK: COMMIT menyimpan perubahan transaksi secara permanen (pesanan 103 tersimpan), sedangkan ROLLBACK membatalkan seluruh perubahan sejak START TRANSACTION (pesanan 104 tidak tersimpan).
- Pentingnya Atomicity & InnoDB: Menjamin operasi multi-tabel (header pesanan, detail barang, stok produk) berhasil utuh atau batal seluruhnya berkat undo/redo log pada engine InnoDB.

5. Analisis Performa Database & Simulasi Jutaan Data dengan EXPLAIN (Tahap 5) [Skor: 25 - Bagian B]

JAWAB:

Pada simulasi >3,1 juta baris data (100.005 pelanggan, 1.007 produk, 1.000.003 pesanan, 2.000.006 detail_pesanan), kami membandingkan EXPLAIN sebelum dan sesudah pembuatan indeks:

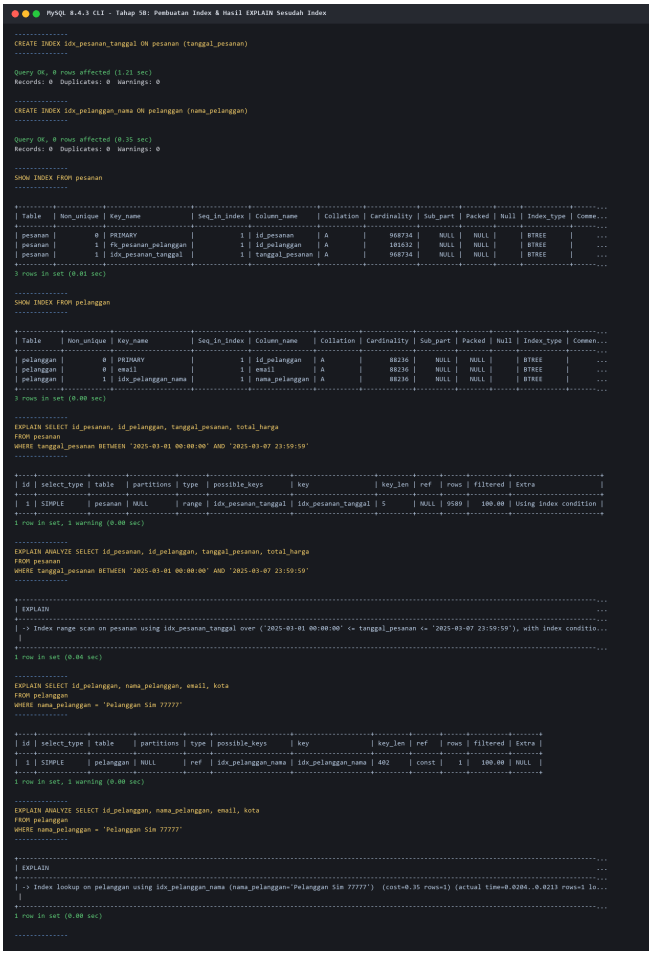
```
EXPLAIN SELECT id_pesanan, id_pelanggan, tanggal_pesanan, total_harga
FROM pesanan
WHERE tanggal_pesanan BETWEEN '2025-03-01 00:00:00' AND '2025-03-07 23:59:59';

CREATE INDEX idx_pesanan_tanggal ON pesanan (tanggal_pesanan);
CREATE INDEX idx_pelanggan_nama ON pelanggan (nama_pelanggan);

EXPLAIN SELECT id_pesanan, id_pelanggan, tanggal_pesanan, total_harga
FROM pesanan
WHERE tanggal_pesanan BETWEEN '2025-03-01 00:00:00' AND '2025-03-07 23:59:59';
```



Gambar 7. Hasil Simulasi 3,1 Juta Baris Data dan EXPLAIN Sebelum Penambahan Index (Full Table Scan)



Gambar 8. Pembuatan Index dan Hasil EXPLAIN Sesudah Penambahan Index (Index Range Scan & Lookup)

- Perbandingan Sebelum vs Sesudah Index: Sebelum indeks, query filter tanggal memindai seluruh tabel (type = ALL, rows = 968.734, waktu 697 ms) dan pencarian nama memindai 88.236 baris (163 ms). Sesudah indeks, metode akses berubah menjadi range (rows = 9.589, waktu 37,3 ms / 18,7x lebih cepat) dan ref (rows = 1, waktu 0,021 ms / ~7.600x lebih cepat).

6. Pemeliharaan Basis Data (Backup, Restore & Git) (Tahap 6) [Skor: 20]

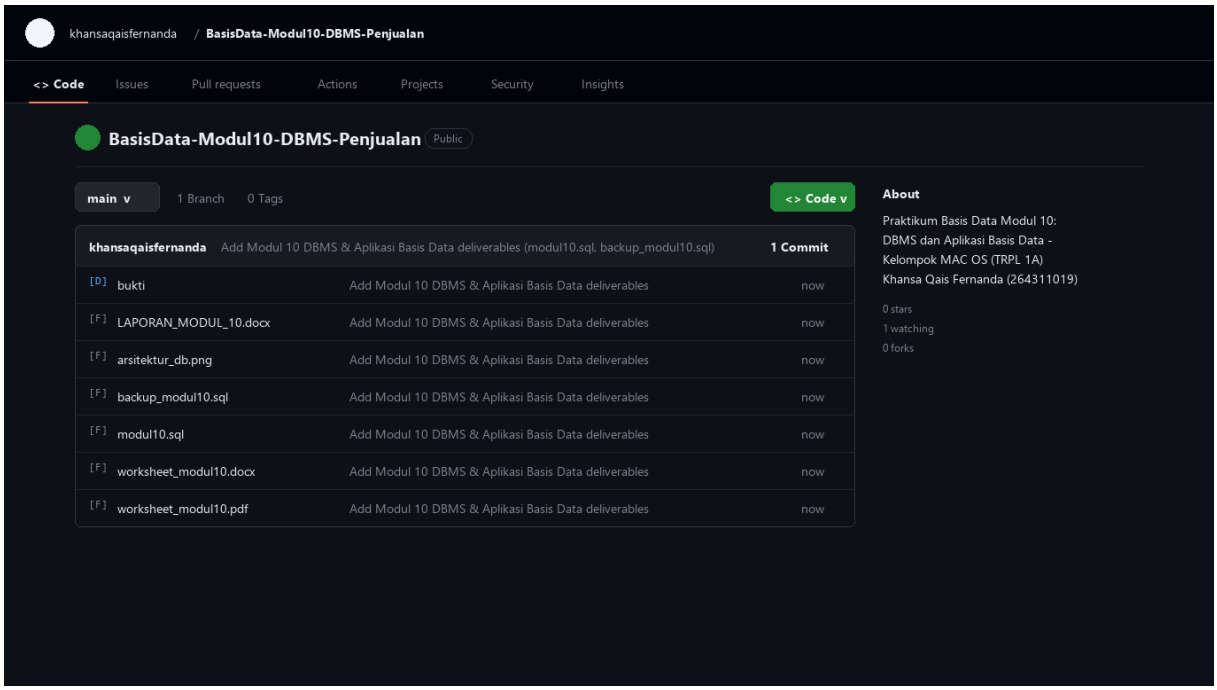
JAWAB:

```
mysqldump -u root --single-transaction --routines --triggers praktikum_modul10 > backup_modul10.sql
mysql -u root -e "CREATE DATABASE praktikum_modul10_restore CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"
mysql -u root praktikum_modul10_restore < backup_modul10.sql
```



Gambar 9. Bukti Ekspor (Backup), Impor (Restore), dan Validasi Jumlah Data Sebelum vs Sesudah Restore

Link GitHub: https://github.com/khansaqaisfernanda/BasisData-Modul10-DBMS-Penjualan



Gambar 10. Bukti Upload (Commit & Push) Berkas modul10.sql dan backup_modul10.sql di GitHub

---

10.4 LEMBAR KERJA (POST-TEST)

1. Instruksi 1 (Bobot 25): Identifikasi DBMS, versi, storage engine, aplikasi pengelola, diagram arsitektur, dan alasan pemilihan!

JAWAB: Menggunakan MySQL Community Server 8.4.3 (Win64, TCP 3306) dengan storage engine InnoDB, dikelola menggunakan phpMyAdmin/HeidiSQL/DBeaver dan MySQL CLI pada lingkungan lokal Laragon (One-Tier Architecture untuk praktikum, Three-Tier Architecture untuk web PjBL). Dipilih karena gratis, mendukung penuh transaksi ACID serta Foreign Key, dan mampu menangani jutaan data.

2. Instruksi 2 (Bobot 30): Script SQL pembuatan tabel beserta PK, FK, hasil pengujian CRUD, dan penjelasan constraint!

JAWAB: Terlampir pada Tahap 2 & 3 (Gambar 3, 4, 5). Primary Key mencegah duplikasi identitas baris (ERROR 1062), Foreign Key (ON DELETE RESTRICT) mencegah transaksi merujuk ke pelanggan/produk fiktif (ERROR 1452) serta mencegah data induk yang memiliki transaksi dihapus sembarangan (ERROR 1451), UNIQUE mencegah email/item ganda, dan CHECK menolak harga/jumlah negatif (ERROR 3819).

3. Instruksi 3 (Bobot 25): Hasil EXPLAIN sebelum dan sesudah pembuatan indeks serta analisis performanya!

JAWAB: Terlampir pada Gambar 7 & 8. Penambahan idx_pesanan_tanggal dan idx_pelanggan_nama mengubah rencana eksekusi dari Full Table Scan (ALL) menjadi Index Range Scan (range) dan Index Lookup (ref), memangkas baris yang diperiksa hingga >99% dan mempercepat eksekusi dari 697 ms menjadi 37,3 ms (18,7x lebih cepat) serta dari 163 ms menjadi 0,021 ms (~7.600x lebih cepat).

4. Instruksi 4 (Bobot 20): Bukti backup dan restore serta alasan pentingnya pemeliharaan dalam DBMS!

JAWAB: Terlampir pada Gambar 9. Backup ke backup_modul10.sql dan restore ke praktikum_modul10_restore terbukti menghasilkan jumlah baris dan CHECKSUM TABLE yang 100% identik. Pemeliharaan dan backup berkala sangat penting untuk mencegah kehilangan data akibat kerusakan disk, human error, maupun gangguan sistem.

10.5 KESIMPULAN

Penerapan storage engine InnoDB beserta constraint relasional (PK, FK, UNIQUE, NOT NULL, CHECK) dan transaksi ACID (COMMIT/ROLLBACK) terbukti menjaga keutuhan dan konsistensi data transaksi penjualan, sementara perancangan indeks B-Tree yang dievaluasi dengan EXPLAIN pada jutaan baris data secara signifikan meningkatkan kecepatan pencarian, dan prosedur backup/restore menjamin ketersediaan data.